

Test Fundamentos Unidad 2

PREGUNTAS

Pregunta 1

En Java, ¿cuál es la diferencia principal entre un tipo primitivo (por ejemplo, int) y su clase envoltorio (Integer)?

- A)** No hay diferencia real, solo cambian las mayúsculas
- B)** El primitivo guarda el valor directamente y es más ligero, mientras que el wrapper es un objeto en el heap con métodos adicionales y posibilidad de ser null
- C)** El primitivo solo se usa en proyectos pequeños y el wrapper en proyectos grandes
- D)** El wrapper siempre es más rápido que el primitivo

Pregunta 2

En un bucle que suma millones de números, ¿qué tipo es más recomendable usar para el acumulador?

- A)** Integer, porque es más “moderno”
- B)** Double, para cualquier suma, aunque sean enteros
- C)** int, para evitar creación de objetos innecesarios y reducir carga al Garbage Collector
- D)** String, para poder mostrar el resultado fácilmente

Pregunta 3

¿Qué afirmación describe mejor el autoboxing y el unboxing en Java?

- A)** Autoboxing es convertir un String en un int, y unboxing es lo contrario
- B)** Autoboxing convierte automáticamente un primitivo en su wrapper, y unboxing convierte el wrapper en primitivo
- C)** Ambos procesos siempre son gratuitos en rendimiento, así que no

hace falta tenerlos en cuenta

D) Solo existen para el tipo int y Integer

Pregunta 4

¿Por qué puede ser peligroso hacer unboxing sobre un objeto Integer que proviene de datos externos?

A) Porque puede lanzar una NullPointerException si el Integer es null

B) Porque convierte el valor a double sin avisar

C) Porque asegura siempre el valor 0 aunque el dato sea otro

D) Porque elimina el objeto del heap automáticamente

Pregunta 5

¿Por qué las colecciones genéricas como List<Integer> necesitan usar wrappers en lugar de tipos primitivos?

A) Porque Java no soporta colecciones de ningún tipo primitivo

B) Porque las colecciones trabajan con referencias a objetos, no con valores primitivos

C) Porque los primitivos no pueden ordenarse

D) Porque los wrappers ocupan menos memoria que los primitivos

Pregunta 6

¿Cuál de las siguientes opciones describe correctamente el uso de stack y heap en la JVM?

A) El stack almacena todos los objetos y el heap solo variables locales

B) El stack guarda variables locales y referencias a objetos, mientras que el heap almacena los objetos en sí

C) El heap es siempre más rápido que el stack

D) El stack se gestiona manualmente y el heap automáticamente

Pregunta 7

¿Cuál es el papel principal del Garbage Collector en Java?

- A)** Optimizar el código eliminando líneas innecesarias
- B)** Buscar y eliminar objetos del heap que ya no tienen referencias activas, liberando memoria
- C)** Ordenar los objetos en memoria alfabéticamente
- D)** Compilar el código fuente a bytecode

Pregunta 8

¿Cuál de las siguientes prácticas ayuda a reducir la presión sobre el heap y el Garbage Collector?

- A)** Crear nuevos objetos dentro de cada iteración de un bucle siempre que sea posible
- B)** Usar StringBuilder para concatenar cadenas en bucles en lugar de usar + continuamente
- C)** Mantener referencias a objetos antiguos "por si acaso"
- D)** Evitar el uso de tipos primitivos

Pregunta 9

¿Qué ejemplo representa una conversión implícita segura (widening) en Java?

- A)** double x = (double) 3.5f;
- B)** int x = (int) 3.9;
- C)** double x = 5;
- D)** byte b = (byte) 200;

Pregunta 10

¿Cuál es el principal riesgo de hacer una conversión explícita de double a int con casting?

- A)** Que el programa no compile nunca
- B)** Que se produzca un redondeo perfecto sin pérdida de información
- C)** Que se pierda la parte decimal (truncamiento) y, en ciertos rangos, incluso se produzca overflow
- D)** Que Java convierta el valor automáticamente a long antes

HOJA DE RESPUESTAS

Pregunta	Respuesta	Justificación
1	B	El primitivo es valor directo y ligero; el wrapper es objeto en heap, con métodos y posibilidad de null
2	C	En bucles intensivos conviene usar int para evitar autoboxing y reducir asignaciones en el heap
3	B	Autoboxing: primitivo → wrapper automático; unboxing: wrapper → primitivo automático
4	A	Si el Integer es null, al convertirlo a int se lanza NullPointerException
5	B	Las colecciones genéricas manejan referencias a objetos, por eso necesitan wrappers como Integer
6	B	El stack guarda variables locales y referencias; el heap almacena los objetos asociados
7	B	El GC libera memoria eliminando objetos sin referencias en el heap
8	B	StringBuilder reduce creación de objetos String intermedios y descarga al GC
9	C	double x = 5; es widening: int → double sin pérdida de información
10	C	El casting double → int trunca decimales y, con valores grandes, puede provocar overflow